C++ Text File I/O & For Loop

Tutorial 8 — File Handling & Loop Mastery

```
#include <fstream>
```

```
for (i=0; i<n; i++)
```

Section 1 — File Input / Output

Key File Stream Classes

`ifstream`
Input file stream — reads from a file

`ofstream`
Output file stream — writes to a file

`fstream`
File stream — both read and write

Required header: `#include <fstream>`

fileio_overview.cpp

```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main() {
  // Open for reading
  ifstream inFile("input.txt");

  // Open for writing
  ofstream outFile("output.txt");

  // Always close when done
  inFile.close();
  outFile.close();
  return 0;
}
```

Exercise 1.1 — Read File · Exercise 1.2 — Write File

Ex 1.1 — Basic File Reading

Read contents of hello.txt and display to console.

read_file.cpp

```
#include <fstream>
#include <iostream>
using namespace std;
int main() {
  ifstream file("hello.txt");
  string line;
  while (getline(file, line))
    cout << line << "\n";
  file.close();
  return 0; }
```

Ex 1.2 — Basic File Writing

Write "Hello, C++ File I/O!" to output.txt.

write_file.cpp

```
#include <fstream>
#include <iostream>
using namespace std;
int main() {
  ofstream file("output.txt");
  file << "Hello, C++ File I/O!";
  file.close();
  cout << "File written!\n";
  return 0;
}
```

Exercise 1.3 — Copy File · Exercise 1.4 — Count Words

Ex 1.3 — Copy File to Another

Read source.txt and write its contents to destination.txt.

copy_file.cpp

```cpp
ifstream src("source.txt");
ofstream dst("destination.txt");

string line;
while (getline(src, line))
  dst << line << "\n";

src.close();
dst.close();
```

Ex 1.4 — Count Words in a File

Count words in a file. Assume words are separated by spaces.

count_words.cpp

```cpp
ifstream file("input.txt");
string word;
int count = 0;

// >> skips whitespace
while (file >> word)
  count++;

cout << "Words: " << count;
file.close();
```

Exercise 1.5 — Replace Words · Exercise 1.6 — Line Counter

Ex 1.5 — Replace Words in File

Replace "apple" with "banana" in story.txt. Save to replaced_story.txt.

replace_words.cpp

```
ifstream in("story.txt");
ofstream out("replaced_story.txt");
string word;

while (in >> word) {
  if (word == "apple")
    word = "banana";
  out << word << " ";
}
```

Ex 1.6 — Line Counter

Read a file and count the total number of lines in it.

line_counter.cpp

```
ifstream file("input.txt");
string line;
int count = 0;

// getline reads full line
while (getline(file, line))
  count++;

cout << "Lines: " << count;
file.close();
```

Section 2 — The For Loop

For Loop Anatomy

A for loop is ideal when you know exactly how many times you want to iterate.

```
for (init ; condition ; update)
{ ... }
```

Init

Runs once — set loop variable
e.g. int i = 0

Condition

Checked before each iteration
e.g. i < 10

Update

Runs after each body execution
e.g. i++

for_loop_example.cpp

```
// Sum 1 to 10
int sum = 0;

for (int i = 1; i <= 10; i++) {
  sum += i;
}

cout << "Sum = " << sum;
// Output: Sum = 55

// Count DOWN example
for (int i = 5; i > 0; i--)
  cout << i << " ";
```

Exercise 2.1 — Natural Numbers · Exercise 2.2 — Factorial

Ex 2.1 — First 10 Natural Numbers

Print the first 10 natural numbers using a for loop.

natural_numbers.cpp

```cpp
for (int i = 1; i <= 10; i++) {
  cout << i << " ";
}

// Output:
// 1 2 3 4 5 6 7 8 9 10
```

Ex 2.2 — Factorial with For Loop

Calculate n! = 1 × 2 × 3 × ... × n using a **for** loop.

factorial_for.cpp

```cpp
int n;
long long factorial = 1;
cout << "Enter n: "; cin >> n;

for (int i = 1; i <= n; i++)
  factorial *= i;

cout << n << "! = " << factorial;
```

Exercise 2.6 — Star Pattern · Exercise 2.7 — Prime Check

Ex 2.6 — Nested Loops: Star Pattern

```
*
**
***
****
```

Uses nested for loops — outer controls rows, inner controls columns.

star_pattern.cpp

```cpp
int rows = 4;

// Outer: controls each row
for (int i = 1; i <= rows; i++) {
  // Inner: prints stars
  for (int j = 1; j <= i; j++)
    cout << "*";
  cout << "\n";
}
```

Ex 2.7 — Prime Number Check

A prime is divisible only by 1 and itself. Check divisors up to √n.

prime_check.cpp

```cpp
int n; cin >> n;
bool isPrime = true;

for (int i = 2; i * i <= n; i++) {
  if (n % i == 0) {
    isPrime = false;
    break;
  }
}
cout << (isPrime ? "Prime" : "Not Prime");
```

Exercise 2.8 — Fibonacci · Exercise 2.9 — Reverse a Number

Ex 2.8 — Fibonacci Sequence

Print first n Fibonacci numbers.

`F(n) = F(n−1) + F(n−2),  F(0)=F(1)=1`

fibonacci.cpp

```
int n; cin >> n;
int a = 1, b = 1;

for (int i = 1; i <= n; i++) {
  cout << a << " ";
  int next = a + b;
  a = b;  b = next;
}
// n=6: 1 1 2 3 5 8
```

Ex 2.9 — Reverse a Number

Reverse the digits of a number.

`e.g. 1234 → 4321`

reverse_num.cpp

```
int n; cin >> n;
int reversed = 0;

// Extract last digit each time
while (n != 0) {
  int digit = n % 10;
  reversed = reversed * 10 + digit;
  n /= 10;
}
cout << reversed;
```

Exercise 2.10 — First Non-Repeated · Exercise 2.11 — Skip Multiples

Ex 2.10 — First Non-Repeated Char

Input: "programming" → Output: "p"
Uses nested for loops.

non_repeated.cpp

```
string s; cin >> s;
for (int i = 0; i < s.size(); i++) {
  bool rep = false;
  for (int j = 0; j < s.size(); j++)
    if (i != j && s[i] == s[j])
      rep = true;
  if (!rep) {
    cout << s[i]; break;
  }
}
```

Ex 2.11 — Skip Multiples (continue)

Print 1–100, skipping multiples of n using **continue**.

skip_multiples.cpp

```
int n = 5;

for (int i = 1; i <= 100; i++) {
  if (i % n == 0) {
    cout << "Skipping: " << i;
    continue;
  }
  cout << i << "\n";
}
```

Exercise 2.13 — Simple Interest Calculator

Problem

Formula:

$SI = P \times R \times T$

P — Principal amount
R — Rate of interest (%)
T — Time in years

Use a for loop to print the total amount (P + SI) at the end of each year.

simple_interest.cpp

```
double P, R;
int T;
cout << "Principal (P): "; cin >> P;
cout << "Rate % (R): ";  cin >> R;
cout << "Years (T): ";   cin >> T;

for (int t = 1; t <= T; t++) {
  double SI = P * (R/100) * t;
  double total = P + SI;
  cout << "Year " << t << ": $" << total;
}
// Final total already printed in loop
```

Key Takeaways

File Reading

ifstream file("name");
getline(file, line) or file >> word
Always call file.close()!

File Writing

ofstream file("name");
file << "text to write";
Creates file if it doesn't exist.

For Loop Syntax

for (init; condition; update)
Ideal when iteration count is known.
Init runs once; update runs each cycle.

break & continue

break — exit the loop immediately
continue — skip to next iteration
Used in Ex 2.10 and Ex 2.11.

Nested For Loops

Use for 2D patterns, grids, tables.
Outer = rows, Inner = columns.
Seen in Ex 2.6 (star pattern).

for vs while

for → known iteration count
while → condition-based (unknown count)
Both repeat until condition is false.